

Examen SEOC CEAMC

Calculatrice basique autorisée
Ordinateurs et autres portables/mobiles interdits

Aucun document autorisé

Rendre chaque partie sur des feuilles séparées

Rendre le sujet

Sujet sur 9 pages

1ère partie

processeurs multicœurs, et leur programmation

Remarque : pour toutes les questions, une explication claire et détaillée est attendue.

Remarque : le barème par exercice est donné à titre indicatif.

Pour tous les exercices dans cet examen, nous allons considérer le système multiprocesseur suivant :

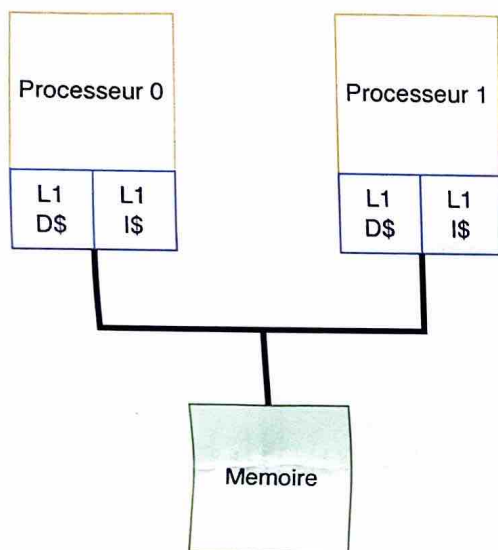


FIGURE 1 : SYSTÈME SUR PUCE CONTENANT DEUX PROCESSEURS DE CALCUL

Address	Data
...	
0xc018	0xa1e7
0xc016	0x64fc
0xc014	0xd154
0xc012	0xf2cf
0xc010	0x3aae
0xc00e	0x224e
0xc00c	0x2da9
0xc00a	0x1c33
0xc008	0x5072
...	
0x200e	0xc3b8
0x200c	0x265e
0x200a	0xbcc4
0x2008	0xfebb
0x2006	0x0000
0x2004	0x0123
0x2002	0xcafe
0x2000	0xdead

FIGURE 2 : CONTENU INITIAL DE LA MÉMOIRE

Le système comprend deux processeurs, chacun avec des caches séparés de premier niveau (L1) pour instructions et données. Les processeurs implémentent des adresses mémoire 16 bits, exécutent des instructions 32 bits (taille d'un mot d'instruction) et opèrent sur des données de 16 bits (taille d'un mot de données). Les deux processeurs (et leur caches) sont interconnectés avec la mémoire au moyen d'un bus partagé.

Le cache de données a les caractéristiques suivantes :

- Capacité de 32 octets
- Chaque ligne de cache contient 4 octets
- Correspondance directe
- Politique d'écriture différée (« write-back »)
- Write-allocate : lors d'une écriture produisant un échec de cache, la ligne est d'abord copiée dans le cache puis modifiée localement
- Afin d'assurer la cohérence entre les caches, les caches de données implémentent le protocole de cohérence MSI à base d'espionnage (« snoop »)

Exercice 1 : multiprocesseurs et cohérence de cache

Maintenant nous allons considérer que les deux processeurs dans le système de la FIGURE 1 sont actifs. Considérez que les caches de données sont vides (toutes leur lignes sont invalides) et que le contenu initial de la mémoire est celui de la FIGURE 2. Vous allez maintenant exécuter chacune des instructions d'accès mémoire dans les exercices ci-dessous, et pour chacune d'entre elles vous devez écrire :

- l'opération du processeur : READ(@0x2002)
 - adresse octet
- les transactions produites sur le bus : <bus, @0x0800, Read Miss>;
 - adresse ligne de cache (ignorer les bits de poids faible désignant le mot)
- le contenu de la mémoire : <mémoire, @0x2003-0x2000, 0xcafe 0xdead>;
- le contenu des caches de données : <cache 0, index 0, étiquette/tag 0x100, état S, données 0xcafe 0xdead>;
 - adresse tag en indiquant uniquement les bits de poids fort correspondants
- le mot transmis au processeur en cas de lecture : <proc, 0xcafe>;

Écrivez uniquement les lignes de cache et les adresses mémoire affectées par l'instruction exécutée. L'état du système résultant après chaque instruction (caches et mémoire) doit être réutilisé pour l'instruction suivante.

Exercice 1.1 (1,5 point) : Processeur 0 : WRITE(@0xc008, 0xf00d)

Exercice 1.2 (1,5 point) : Processeur 1 : WRITE(@0xc00a, 0xbabe)

Exercice 1.3 (1,5 point) : Processeur 0 : READ (@0xc00a)

Exercice 1.4 (1,5 points) : Processeur 0 : READ (@0x2004)

Exercice 1.5 (1,5 points) : Processeur 1 : WRITE(@0x2006, 0x4567)

Exercice 2 : instructions atomiques

La routine `add_modulo` permet d'incrémenter par une valeur n , le contenu de l'adresse mémoire v , modulo une valeur m , c'est à dire, $*v = (*v + n) \% m$. La fonction retourne la valeur avant la modification.

Le processeur RISC-V cible n'implémente pas l'extension « M », donc les instructions de multiplication, division ou modulo ne sont pas disponibles.

Le prototype de la fonction est le suivant :

```
unsigned int add_modulo(unsigned int *v, unsigned int n, unsigned int m)
```

Cette fonction peut-être exécutée par plusieurs processeurs simultanément et sur la même adresse v . Vous devez donc assurer que les modifications soient réalisées de manière atomique.

Exercice 2.1 (3 points) : Proposez une routine en assembleur RISC-V permettant d'implémenter la fonction `add_modulo`. Rappel : les instructions de multiplication, division ou modulo ne sont pas disponibles.

RISC-V

Reference Data

RV64I BASE INTEGER INSTRUCTIONS, in alphabetical order			NOTE
MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)
add, addw	R	ADD (Word)	$R[rd] = R[rs1] + R[rs2]$
addi, addiw	I	ADD Immediate (Word)	$R[rd] = R[rs1] + imm$
and	R	AND	$R[rd] = R[rs1] \& R[rs2]$
andi	I	AND Immediate	$R[rd] = R[rs1] \& imm$
auipc	U	Add Upper Immediate to PC	$R[rd] = PC + (imm, 12b0)$
beq	S	Branch Equal	$if(R[rs1] == R[rs2]) PC = PC + (imm, 12b0)$
bge	S	Branch Greater than or Equal	$if(R[rs1] >= R[rs2]) PC = PC + (imm, 12b0)$
bgeu	S	Branch $\geq$ Unsigned	$if(R[rs1] >=u R[rs2]) PC = PC + (imm, 12b0)$
blt	S	Branch Less Than	$if(R[rs1] < R[rs2]) PC = PC + (imm, 12b0)$
bltu	S	Branch Less Than Unsigned	$if(R[rs1] <u R[rs2]) PC = PC + (imm, 12b0)$
bne	S	Branch Not Equal	$if(R[rs1] != R[rs2]) PC = PC + (imm, 12b0)$
csrrc	I	Cont./Stat.RegRead&Clear	$R[rd] = CSR; CSR = CSR \& \sim R[rs1]$
csrrci	I	Cont./Stat.RegRead&Clear Imm	$R[rd] = CSR; CSR = CSR \& \sim imm$
csrrs	I	Cont./Stat.RegRead&Set	$R[rd] = CSR; CSR = CSR R[rs1]$
csrrsi	I	Cont./Stat.RegRead&Set Imm	$R[rd] = CSR; CSR = CSR imm$
csrrw	I	Cont./Stat.RegRead&Write	$R[rd] = CSR; CSR = R[rs1]$
csrrwi	I	Cont./Stat.RegRead&Write Imm	$R[rd] = CSR; CSR = imm$
ebreak	I	Environment BREAK	Transfer control to debugger
ecall	I	Environment CALL	Transfer control to operating system
fence	I	Synch thread	Synchronizes threads
fence.i	I	Synch Instr & Data	Synchronizes writes to instruction stream
jal	UI	Jump & Link	$R[rd] = PC + 4; PC = PC + (imm, 12b0)$
jalr	I	Jump & Link Register	$R[rd] = PC + 4; PC = R[rs1] + imm$
lb	I	Load Byte	$R[rd] = (56b0M[R[rs1] + imm](7:0))$
lbub	I	Load Byte Unsigned	$R[rd] = (56b0M[R[rs1] + imm](7:0))$
ld	I	Load Doubleword	$R[rd] = M[R[rs1] + imm](63:0)$
lh	I	Load Halfword	$R[rd] = (48b0M[R[rs1] + imm](15:0))$
lhu	I	Load Halfword Unsigned	$R[rd] = (48b0M[R[rs1] + imm](15:0))$
lui	U	Load Upper Immediate	$R[rd] = (32bimm<31>, imm, 12b0)$
lw	I	Load Word	$R[rd] = (32b0M[R[rs1] + imm](31:0))$
lwu	I	Load Word Unsigned	$R[rd] = (32b0M[R[rs1] + imm](31:0))$
or	R	OR	$R[rd] = R[rs1] R[rs2]$
ori	I	OR Immediate	$R[rd] = R[rs1] imm$
sb	S	Store Byte	$M[R[rs1] + imm](7:0) = R[rs2](7:0)$
sd	S	Store Doubleword	$M[R[rs1] + imm](63:0) = R[rs2](63:0)$
sh	S	Store Halfword	$M[R[rs1] + imm](15:0) = R[rs2](15:0)$
sll, sllw	R	Shift Left (Word)	$R[rd] = R[rs1] << R[rs2]$
slli, slliw	I	Shift Left Immediate (Word)	$R[rd] = R[rs1] << imm$
slt	R	Set Less Than	$R[rd] = (R[rs1] < R[rs2]) ? 1 : 0$
slti	I	Set Less Than Immediate	$R[rd] = (R[rs1] < imm) ? 1 : 0$
sltiu	I	Set < Immediate Unsigned	$R[rd] = (R[rs1] <u imm) ? 1 : 0$
sltu	R	Set Less Than Unsigned	$R[rd] = (R[rs1] <u R[rs2]) ? 1 : 0$
sra, sraw	R	Shift Right Arithmetic (Word)	$R[rd] = R[rs1] >> R[rs2]$
srai, srawi	I	Shift Right Arith Imm (Word)	$R[rd] = R[rs1] >> imm$
srl, srlw	R	Shift Right (Word)	$R[rd] = R[rs1] >> R[rs2]$
srli, srlwi	I	Shift Right Immediate (Word)	$R[rd] = R[rs1] >> imm$
sub, subw	R	SUBtract (Word)	$R[rd] = R[rs1] - R[rs2]$
sw	S	Store Word	$M[R[rs1] + imm](31:0) = R[rs2](31:0)$
xor	R	XOR	$R[rd] = R[rs1] \wedge R[rs2]$
xori	I	XOR Immediate	$R[rd] = R[rs1] \wedge imm$

- Notes: 1) The Word version only operates on the rightmost 32 bits of a 64-bit registers
 2) Operation assumes unsigned integers (instead of 2's complement)
 3) The least significant bit of the branch address in jalr is set to 0
 4) (signed) Load instructions extend the sign bit of data to fill the 64-bit register
 5) Replicates the sign bit to fill in the leftmost bits of the result during right shift
 6) Multiply with one operand signed and one unsigned
 7) The Single version does a single-precision operation using the rightmost 32 bits of a 64-bit F register
 8) Classify writes a 10-bit mask to show which properties are true (e.g., -inf, -0, +0, +inf, denorm, ...)
 9) Atomic memory operation; nothing else can interpose itself between the read and the write of the memory location
 The immediate field is sign-extended in RISC-V

ARITHMETIC CORE INSTRUCTION SET

RV64M Multiply Extension

MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
mul, mulw	R	MULTiply (Word)	$R[rd] = R[rs1] * R[rs2](63:0)$	1)
mulh	R	MULTiply High	$R[rd] = (R[rs1] * R[rs2])(127:64)$	2)
mulhu	R	MULTiply High Unsigned	$R[rd] = (R[rs1] *u R[rs2])(127:64)$	6)
mulhsu	R	MULTiply upper Half Sign/Uns	$R[rd] = (R[rs1] * R[rs2])(127:64)$	1)
div, divw	R	DIVide (Word)	$R[rd] = R[rs1] / R[rs2]$	2)
divu	R	DIVide Unsigned	$R[rd] = (R[rs1] /u R[rs2])$	1)
rem, remw	R	REMAinder (Word)	$R[rd] = (R[rs1] \% R[rs2])$	1,2)
remu, remuw	R	REMAinder Unsigned (Word)	$R[rd] = (R[rs1] \%u R[rs2])$	

RV64F and RV64D Floating-Point Extensions

MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
fld, flw	I	Load (Word)	$F[rd] = M[R[rs1] + imm]$	1)
fsw, fsw	S	Store (Word)	$M[R[rs1] + imm] = F[rd]$	1)
fadd, fadd.d	R	ADD	$F[rd] = F[rs1] + F[rs2]$	7)
fsub, fsub.d	R	SUBtract	$F[rd] = F[rs1] - F[rs2]$	7)
fmul, fmul.d	R	MULTiply	$F[rd] = F[rs1] * F[rs2]$	7)
fdiv, fdiv.d	R	DIVide	$F[rd] = F[rs1] / F[rs2]$	7)
fsqrt, fsqrt.d	R	SQuare RooT	$F[rd] = \sqrt{F[rs1]}$	7)
fmaadd, fmaadd.d	R	Multiply-ADD	$F[rd] = F[rs1] * F[rs2] + F[rs3]$	7)
fmsub, fmsub.d	R	Multiply-SUBtract	$F[rd] = F[rs1] * F[rs2] - F[rs3]$	7)
fnmadd, fnmadd.d	R	Negative Multiply-ADD	$F[rd] = -F[rs1] * F[rs2] + F[rs3]$	7)
fnmsub, fnmsub.d	R	Negative Multiply-SUBtract	$F[rd] = -F[rs1] * F[rs2] - F[rs3]$	7)
fsgnj, fsgnj.d	R	SIGN source	$F[rd] = \{ F[rs2] < 63 >, F[rs1] < 62:0 > \}$	7)
fsgnjn, fsgnjn.d	R	Negative SIGN source	$F[rd] = \{ \sim F[rs2] < 63 >, F[rs1] < 62:0 > \}$	7)
fsgnjx, fsgnjx.d	R	Xor SIGN source	$F[rd] = \{ F[rs2] < 63 > ^ F[rs1] < 62:0 >, F[rs1] < 62:0 > \}$	7)
fmin, fmin.d	R	MINimum	$F[rd] = (F[rs1] < F[rs2]) ? F[rs1] : F[rs2]$	7)
fmax, fmax.d	R	MAXimum	$F[rd] = (F[rs1] > F[rs2]) ? F[rs1] : F[rs2]$	7)
feq, feq.d	R	Compare Float Equal	$R[rd] = (F[rs1] == F[rs2]) ? 1 : 0$	7)
flt, flt.d	R	Compare Float Less Than	$R[rd] = (F[rs1] < F[rs2]) ? 1 : 0$	7)
fle, fle.d	R	Compare Float Less than or =	$R[rd] = (F[rs1] <= F[rs2]) ? 1 : 0$	7)
fclass, fclass.d	R	Classify Type	$R[rd] = \text{class}(F[rs1])$	7,8)
fmv, fmv.d.x	R	Move from Integer	$F[rd] = R[rs1]$	7)
fmv, fmv.x.d	R	Move to Integer	$R[rd] = F[rs1]$	7)
fcvt, fcvt.d	R	Convert to SP from DP	$F[rd] = \text{single}(F[rs1])$	7)
fcvt, fcvt.d.s	R	Convert to DP from SP	$F[rd] = \text{double}(F[rs1])$	7)
fcvt, fcvt.s.w	R	Convert from 32b Integer	$F[rd] = \text{float}(R[rs1])(31:0)$	7)
fcvt, fcvt.s.l	R	Convert from 64b Integer	$F[rd] = \text{float}(R[rs1])(63:0)$	7)
fcvt, fcvt.s.wu	R	Convert from 32b Int Unsigned	$F[rd] = \text{float}(R[rs1])(31:0)$	2,7)
fcvt, fcvt.s.lu	R	Convert from 64b Int Unsigned	$F[rd] = \text{float}(R[rs1])(63:0)$	2,7)
fcvt, fcvt.w.d	R	Convert to 32b Integer	$R[rd](31:0) = \text{integer}(F[rs1])$	7)
fcvt, fcvt.l.s	R	Convert to 64b Integer	$R[rd](63:0) = \text{integer}(F[rs1])$	7)
fcvt, fcvt.wu.s	R	Convert to 32b Int Unsigned	$R[rd](31:0) = \text{integer}(F[rs1])$	2,7)
fcvt, fcvt.lu.s	R	Convert to 64b Int Unsigned	$R[rd](63:0) = \text{integer}(F[rs1])$	2,7)

RV64A Atomic Extension

MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
amoadd, amoadd.d	R	ADD	$R[rd] = M[R[rs1]]$	9)
amoand, amoand.d	R	AND	$M[R[rs1]] = M[R[rs1]] \& R[rs2]$	9)
amomax, amomax.d	R	MAXimum	$R[rd] = M[R[rs1]]$	9)
amomaxu, amomaxu.d	R	MAXimum Unsigned	$M[R[rs1]] = M[R[rs1]] \& R[rs2]$	9)
amomin, amomin.d	R	MINimum	$R[rd] = M[R[rs1]]$	9)
amominu, amominu.d	R	MINimum Unsigned	$M[R[rs1]] = M[R[rs1]] \& R[rs2]$	9)
amoor, amoor.d	R	OR	$R[rd] = M[R[rs1]]$	9)
amoswap, amoswap.d	R	SWAP	$M[R[rs1]] = M[R[rs1]] \wedge R[rs2]$	9)
amoxor, amoxor.d	R	XOR	$R[rd] = M[R[rs1]]$	9)
lr, lr.d	R	Load Reserved	$R[rd] = M[R[rs1]]$	9)
sc, sc.d	R	Store Conditional	$R[rd] = M[R[rs1]]$	9)

CORE INSTRUCTION FORMATS

	31	27	26	25	24	20	19	15	14	12	11	7	6	0	
R	funct7					rs2					rs1				
I	imm[11:0]					rs1					funct3				
S	imm[11:5]					rs2					rs1				
SB	imm[12[0:5]					rs2					rs1				
U	imm[31:12]										funct3				
UI	imm[20[10:11][19:12]										imm[4:1[1]				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				
											rd				

PSEUDO INSTRUCTIONS

MNEMONIC	NAME	DESCRIPTION	USES
bneqz	Branch = zero	$if(R[rs1] == 0) PC = PC + (imm, lb0)$	bneq
bneqz	Branch # zero	$if(R[rs1] != 0) PC = PC + (imm, lb0)$	bneq
fabsw.s, fabsw.d	Absolute Value	$F[rd] = F[rs1] < 0 ? -F[rs1] : F[rs1]$	fabsw
fmsw.s, fmsw.d	FP Move	$F[rd] = F[rs1]$	fmsw
fnegw.s, fnegw.d	FP negate	$F[rd] = -F[rs1]$	fnegw
j	Jump	$PC = (imm, lb0)$	j
jr	Jump register	$PC = R[rs1]$	jr
la	Load address	$R[rd] = \text{address}$	la
li	Load imm	$R[rd] = \text{imm}$	li
mv	Move	$R[rd] = R[rs1]$	mv
neg	Negate	$R[rd] = -R[rs1]$	neg
nop	No operation	$R[rd] = R[rd]$	nop
not	Not	$R[rd] = \sim R[rs1]$	not
ret	Return	$PC = R[ra]$	ret
sewz	Set = zero	$PC = (R[rs1] == 0) ? 1 : 0$	sewz
snez	Set # zero	$R[rd] = (R[rs1] == 0) ? 1 : 0$	snez

OPCODES IN NUMERICAL ORDER BY OPCODE

MNEMONIC	FMTC	OPCODE	FUNCT3	FUNCT7 OR IMM	HEXADCEMAL
lb	I	0000011	000		03/0
lh	I	0000011	001		03/1
lw	I	0000011	010		03/2
ld	I	0000011	011		03/3
lbu	I	0000011	100		03/4
lbu	I	0000011	101		03/5
lbu	I	0000011	110		03/6
fence	I	0001111	000		0F/0
fence.i	I	0001111	001		0F/1
addi	I	0010011	000		13/0
slli	I	0010011	001	0000000	13/1/00
slli	I	0010011	010		13/2
slli	I	0010011	011		13/3
xori	I	0010011	100		13/4
xori	I	0010011	101	0000000	13/5/00
srai	I	0010011	101	0100000	13/5/20
ori	I	0010011	110		13/6
andi	I	0010011	111		13/7
auipc	U	0010111	000		17
addiw	I	0010111	000		18/0
addiw	I	0010111	001	0000000	18/1/00
srlw	I	0010111	101	0000000	18/5/00
srlw	I	0010111	101	0100000	18/5/20
sb	S	0100011	000		23/0
sh	S	0100011	001		23/1
sw	S	0100011	010		23/2
sd	S	0100011	011		23/3
add	R	0110011	000	0000000	33/0/00
sub	R	0110011	000	0100000	33/0/20
sll	R	0110011	001	0000000	33/1/00
sll	R	0110011	010	0000000	33/2/00
altu	R	0110011	011	0000000	33/3/00
xor	R	0110011	100	0000000	33/4/00
sri	R	0110011	101	0000000	33/5/00
sra	R	0110011	101	0100000	33/5/20
or	R	0110011	110	0000000	33/6/00
and	R	0110011	111	0000000	33/7/00
lui	U	0110111	000		37
addw	R	0110111	000	0000000	38/0/00
subw	R	0110111	000	0100000	38/0/20
sllw	R	0110111	001	0000000	38/1/00
srlw	R	0110111	101	0000000	38/5/00
sraw	R	0110111	101	0100000	38/5/20
beq	SB	1100011	000		63/0
bne	SB	1100011	001		63/1
blt	SB	1100011	100		63/4
bge	SB	1100011	101		63/5
bltu	SB	1100011	110		63/6
bgeu	SB	1100011	111		63/7
jalc	I	1100111	000		6F
jalc	I	1101111	000		6F
ecall	I	1110011	000	000000000000	73/0/000
ebreak	I	1110011	000	000000000001	73/0/001
CSRWR	I	1110011	001		73/1
CSRWR	I	1110011	010		73/2
CSRWR	I	1110011	011		73/3
CSRWR	I	1110011	101		73/5
CSRWR	I	1110011	110		73/6
CSRWR	I	1110011	111		73/7

③

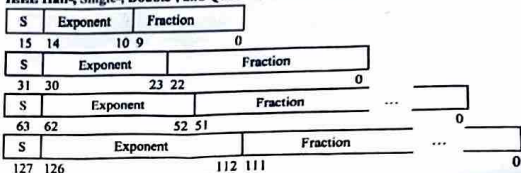
REGISTER NAME, USE, CALLING CONVENTION

④

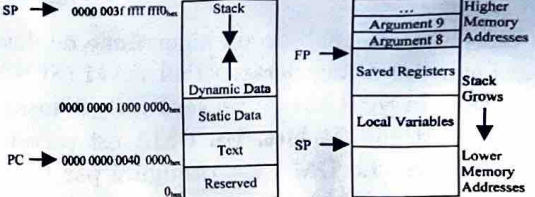
REGISTER	NAME	USE	SAVER
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Caller
x3	gp	Global pointer	Caller
x4	tp	Thread pointer	Caller
x5-x7	t0-t2	Temporaries	Caller
x8	s0/s1	Saved register/Frame pointer	Caller
x9	a1	Saved register	Caller
x10-x11	a0-a1	Function arguments/Return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s7	Saved registers	Caller
x28-x31	t3-t6	Temporaries	Caller
x0-x7	t0-t7	FP Temporaries	Caller
x8-x9	s0-s1	FP Saved registers/Return values	Caller
x10-x11	a0-a1	FP Function arguments/Return values	Caller
x12-x17	a2-a7	FP Function arguments	Caller
x18-x27	s2-s7	FP Saved registers	Caller
x28-x31	t3-t6	FP Saved registers	Caller
x0-x7	t0-t7	$R[rd] = R[rs1] + R[rs2]$	Caller

IEEE 754 FLOATING-POINT STANDARD

$(-1)^s \times (1 + \text{Fraction}) \times 2^{\text{Exponent} - \text{Bias}}$
 where Half-Precision Bias = 15, Single-Precision Bias = 127,
 Double-Precision Bias = 1023, Quad-Precision Bias = 16383
 IEEE Half-, Single-, Double-, and Quad-Precision Formats:



MEMORY ALLOCATION



SIZE PREFIXES AND SYMBOLS

SIZE	PREFIX	SYMBOL	SIZE	PREFIX	SYMBOL
10 ³	Kilo-	K	2 ¹⁰	Kibi-	Ki
10 ⁶	Mega-	M	2 ²⁰	Mebi-	Mi
10 ⁹	Giga-	G	2 ³⁰	Gibi-	Gi
10 ¹²	Tera-	T	2 ⁴⁰	Tebi-	Ti
10 ¹⁵	Peta-	P	2 ⁵⁰	Pebi-	Pi
10 ¹⁸	Exa-	E	2 ⁶⁰	Exbi-	Ei
10 ²¹	Zetta-	Z	2 ⁷⁰	Zebi-	Zi
10 ²⁴	Yotta-	Y	2 ⁸⁰	Yobi-	Yi
10 ²⁷	milli-	m	10 ⁻¹⁵	femto-	f
10 ³⁰	micro-	μ	10 ⁻¹⁸	atto-	a
10 ³³	nano-	n	10 ⁻²¹	zepto-	z
10 ³⁶	pico-	p	10 ⁻²⁴	yocto-	y

2eme partie

Gestion de la mémoire

2 AAA pour la détection d'objet

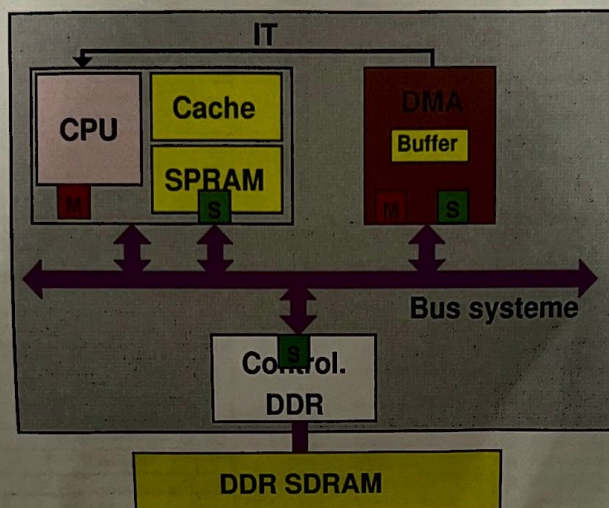


FIGURE 1 – Architecture mémoire

On souhaite réaliser un algorithme de détection de visage à l'aide d'un processeur 32 bits qui dispose d'un cache et d'une Scratch-Pad RAM (SPRAM) ainsi qu'illustré sur la figure 1. Le processeur accède au cache et à la SPRAM en un seul cycle d'horloge. Le processeur est connecté à la mémoire externe à travers un bus système 64 bits. Un DMA est présent dans le système afin de réaliser différents transferts entre les mémoires. Le DMA est configuré par l'adresse de départ, l'adresse d'arrivée et la quantité de données transférées. Le DMA émet une interruption lorsque le transfert est complètement terminé. Afin que le DMA stocke les données d'un burst entre la lecture des données et leur écriture, le DMA contient une mémoire dont la taille est celle du burst maximum.

2.1 Fonctionnement du DMA

► Question 1 :

Rappeler en deux phrases la différence entre une SPRAM et un cache.

Paramètres système :

- Le bus système est un bus 64 bits
- $b_{\max}$ est la taille maximum d'un burst sur le bus système (exprimé en octets). Un burst est constitué d'une séquence de mots de 64 bits
- l_m la latence d'accès en lecture à la mémoire externe depuis un maître sur le bus
- l_{SPRAM} la latence d'accès en écriture vers la SPRAM depuis un maître sur le bus
- t_{IT} est le temps minimum de réponse du processeur à une interruption. Cela correspond au nombre de cycles minimum entre le moment où une interruption apparaît et l'exécution de la première instruction de la routine de traitement d'interruption. Ce temps inclus la sauvegarde de contexte, etc...

▷ Question 2 :

Dessiner un chronogramme type pour le transfert d'un paquet de n KiOctets depuis la mémoire externe vers la SPRAM. Sur ce chronogramme :

- Dessiner le chronogramme d'une séquence de transfert DMA d'un burst de taille $b_{\max}$ provenant de la mémoire externe et écrites en SPRAM
- Montrer comment synchroniser un DMA avec le logiciel en utilisant une interruption

Finalement, donner l'équation qui exprime le nombre de cycles minimum pour ce transfert, en fonction de n , l_{SPRAM} , l_m , $b_{\max}$ et t_{IT}

2.2 Application à la détection de visage

On souhaite réaliser un algorithme de détection de visage dans une image stockée en mémoire externe. Un pixel de l'image est codé sur 8 bits. L'image d'entrée I est de taille $I_x \times I_y$.

Les pixels sont stockés en mémoire dans l'ordre canonique. C'est-à-dire que l'adresse d'un pixel de coordonnée (x, y) de l'image I , noté $I(x, y)$, se calcule par :

$$\text{adresse}(I(x, y)) = x + y * I_x$$

Avec x variant de 0 à $I_x - 1$ (inclue) et y variant de 0 à $I_y - 1$. Le pixel $(0, 0)$ est en haut à gauche et le pixel $(I_x - 1, I_y - 1)$ est en bas à droite.

On notera qu'une zone $z = z_x \times z_y$ de l'image I a une largeur de z_x pixels et une hauteur de z_y pixels. La notation $(c_x, c_y) + z_x \times z_y$ désigne une zone de coin supérieur gauche de coordonnée (c_x, c_y) et de taille $z_x \times z_y$.

Valeurs numériques

Dans la suite :

- $I_x = I_y = 256$
- $l_m = 27$ cycles
- $l_{\text{SPRAM}} = 6$ cycles
- $b_{\max} = 128$ Octets
- $t_{\text{IT}} = 856$ cycles
- La SPRAM fait 32 KiOctets

L'algorithme de détection de visage commence par calculer l'image intégrale de l'image originale (en provenance de la caméra). Un pixel de l'image intégrale $II(x, y)$ prend pour valeur la somme de tous les pixels du rectangle compris entre l'origine (en haut à gauche) et le pixel $I(x, y)$ de l'image originale.

Afin de savoir si un visage est détecté dans une zone de taille $d_x \times d_y$ pixels de coin supérieur gauche (c_x, c_y) , on a besoin de tous les pixels de l'image intégrale de la zone $(c_x, c_y) + d_x \times d_y$. Pratiquement le détecteur a une taille $d_x \times d_y = 24 \times 24$.

Le calcul de l'image intégrale se fait en deux passes : une intégrale verticale suivie d'une intégrale horizontale. On exécute l'algorithme suivant :

```
/* Entrée : image I de taille Ix*Iy */
/* Sortie : image II de taille Ix*Iy */
/* 1ere passe : intégrale verticale */
for(x=0; x<Ix; x++) /* pour chaque colonne */
{
    II(x,0)= I(x, 0); /* initialise l'intégrale */
    for(y=1; y<Iy; y++) /* pour chaque ligne */
        II(x,y)= II(x,y-1) + I(x,y);
```

```

}
/* 2nde passe : intégrale horizontale */
for(y=0; y<Iy; y++)          /* pour chaque ligne */
  for(x=1; x<Ix; x++)          /* pour chaque colonne */
    II(x,y)= II(x-1,y) + II(x,y);

```

▷ **Question 3 :**

Si l'image originale est composée de pixels codés sur 8 bits, combien de bits sont nécessaires pour coder les pixels de l'image intégrale ?

Indications :

- Donner l'équation théorique du nombre de bits à ajouter, en considérant le pire-cas
- Donner la valeur du nombre de bits en fonction des paramètres précédents
- Prendre une valeur parmi 8, 16 ou 32 bits

▷ **Question 4 :**

Quelle est la taille de la zone mémoire occupée par l'image intégrale ?

Nous constatons qu'il n'est pas possible de stocker entièrement l'image originale ni l'image intégrale elle-même en SPRAM pour réaliser ce calcul. En conséquence des optimisations sont nécessaires pour travailler avec la SPRAM.

En tout premier lieu, nous constatons aussi que l'image originale I doit être chargée par lignes et par colonnes pour calculer les lignes et colonnes de l'image intégrale II .

▷ **Question 5 :**

- Combien de temps minimum faut-il pour charger une *ligne* de $l_x \times 1$ pixels dans la SPRAM ?
- Combien de temps minimum faut-il pour charger une *colonne* de $1 \times c_y$ pixels dans la SPRAM ?

Indication : Attention au schéma d'adressage car les pixels d'une colonne ne sont pas à des adresses contiguës

▷ **Question 6 :**

Combien de lignes et colonnes l'algorithme précédent nécessite-t-il de charger ?

- Dans la passe verticale
- Dans la passe horizontale

En déduire le temps total passé à charger les données pour le calcul de l'image intégrale.

2.3 Optimisations du détecteur

On souhaite optimiser la détection de visage en minimisant les transferts mémoire. Dans cet objectif on réalise la détection de visage par tuiles de l'image et on déplace progressivement l'origine de ces tuiles pour balayer toute l'image d'origine.

Dans cette situation, le détecteur de taille $d_x \times d_y$ est appliqué à tous les pixels d'une tuile de l'image, chaque tuile étant virtuellement considérée comme une image. L'image intégrale de la tuile est calculée directement à partir des pixels d'origine correspondant à cette tuile et il n'est plus nécessaire de calculer l'image intégrale complète.

▷ **Question 7 :**

Pour appliquer le détecteur à toutes les zones $d_x \times d_y$ dont les origines sont dans une tuile $T = (t_x, t_y) + T_x \times T_y$, quelle est la zone Z de l'image originale nécessaire ?

Application numérique avec $T_A = T_x \times T_y = 16 \times 16$ puis $T_B = T_x \times T_y = 32 \times 8$

Pour optimiser le système seule l'image intégrale de la zone Z est calculée. En effet, l'algorithme de détection utilise l'image intégrale pour calculer des différences et il donne le même résultat soit que l'on utilise l'image intégrale de toute l'image ou bien soit seulement celle calculée sur la zone Z .

▷ **Question 8 :**

De quelle quantité de mémoire a-t-on besoin pour stocker dans la SPRAM la zone de pixels Z et l'image intégrale correspondante ?

Application numérique avec les valeurs précédentes pour les cas T_A et T_B

▷ **Question 9 :**

Combien de temps faut-il pour charger en SPRAM les pixels de la zone Z ?

Application numérique avec les valeurs précédentes pour T_A et T_B

▷ **Question 10 :**

En considérant que toute l'image est décomposée en tuiles de taille T , calculer le nombre total de burst et le temps total de chargement pour effectuer la détection sur toute l'image.

Application numérique avec les valeurs précédentes pour T_A et T_B

2.4 Optimisation de la mémoire

Afin d'optimiser encore le système précédent, on cherche à réduire le nombre de bursts en modifiant l'organisation des données en mémoire.

Les données de l'image d'origine sont maintenant stockées en blocs de taille $b = b_x \times b_y$. Les blocs sont de taille $b = 16 \times 16$.

Tous les pixels d'un bloc sont à des adresses consécutives et un bloc peut donc être chargée par un nombre réduit de bursts.

▷ **Question 11 :**

Donner le nombre de burst et le temps nécessaire pour charger un bloc en SPRAM depuis la DDR.

▷ **Question 12 :**

Pour réaliser le traitement d'une tuile T , combien de blocs sont nécessaires ?

Calculer le nombre total de burst et le temps total de chargement pour effectuer la détection sur toute l'image.

▷ **Question 13 :**

Peut-on réutiliser des blocs lorsque l'on passe d'une tuile à une autre ? Donner le gain apporté par une stratégie de réutilisation efficace